

writer lock may be a useful pattern to increase the concurrency among concurrent readers.

Note that simple reader/writer locks can lead to situations where the writer is starved as more and more readers enter the critical section. The writer is prevented from entering the critical section in the presence of a reader. In order to avoid starvation of the writer, biased reader/writer locks are used, wherein no further readers are allowed to enter the critical section if a writer is already waiting to enter the critical section.

TOCTTOU

Our sixth question in last month's column was on the TOCTTOU (Time Of Creation To Time Of Use) race condition, which can happen in single threaded code also.

Consider the following code snippet:

```
if (access(my_file, RW_OK) != 0) {
    exit(1);
}

my_fd = open(my_file, O_RDWR);

// do something with my_fd...
```

The code uses the `access()` system call to check if the person running the application has the necessary permission to access the specified file, 'my_file' before it opens the file and performs further operations on it. If the user has the correct permissions, then the 'access' system call returns zero. Else it exits with an error.

However, it is possible for a malicious user to make 'my_file' point to a symbolic link of a totally different file after the access system call succeeds. For example, 'my_file' can be made to point to `/etc/passwd` file so that the malicious user gains access to update the password file. Since the access permission check of the file and subsequent use of it does not happen automatically, it is possible for the malicious user to use the window in between the time of check and the time of use to replace the

original file with a different file. This is an example of a file race condition known as TOCTTOU.

Locking a linked list

The eighth question in last month's column was on parallelising the access to a singly linked list by multiple threads, where most of the operations involve searching the list, with infrequent updates. I had asked the readers regarding the choice of locking granularity for this data structure.

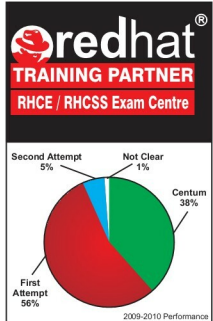
One of the readers (Sukesh Kumar) had suggested using an in-object flag, which can be used to do co-operative locking among the threads. He wrote, "If a linked list has to be accessed by multiple threads, it does not make sense to have a single lock to access the linked list. If the lock is provided at the data level, the locks count will be huge. The solution can be achieved by having a transactional mechanism to update the linked list.

"Let's have a single copy of the linked list for every thread to access. The moment someone tries to update data in some node, take a copy of the node in temp and MARK the original node in the linked list (or lock the node so that no one else can write on it). Once the thread updates the data, replicate the whole data part into MARKed node, and UNMARK it."

Instead of providing the exact solution, I would like to ask readers to answer the following two questions: What are the issues you see with the above approach? What would you do if you are told that you cannot change the layout of a node in the linked list?

Dynamic memory allocation in multi-threaded code

Our ninth question in last month's column was on dynamic memory allocation in a multi-threaded code. Consider the same problem of parallelising a linked list operation. Now there are frequent insertions and deletion of nodes to the linked list by different threads. What are the design considerations you need to keep in mind while designing a dynamic memory allocator for such an application?



At ADVANTAGE PRO, we do not make tall claims but produce 99% results month after month – TAMIL NADU'S NO. 1 PERFORMING REDHAT PARTNER

- Pioneer in LINUX training since 1999.
- Chennai's #1 RHCE training Centre
- 100% results every month.
- Many centum students.
- Also get trained on My SQL - CMDBA, CMDEV, PHP, Perl, Ruby, Python, Ajax...

**Next RHCE/RHCSS Exam
10th & 24th May 2010**

**"Do Not Wait! Be a Part
of the Winning Team"**

VECTRA TECHNOLOGIES
IT TRAINING DIVISION OF
Vectra Technologies Pvt. Ltd.
Open Source Academy

**Regd. Off: Wing 1 & 2, IV Floor,
Jhaver Plaza, 1A, N.H. Road,
Nungambakkam, Chennai - 34.**

Ph : 28263529 / 3530 / 3540

Telefax : 28263527

**E-mail : enquiry@vectratech.in
www.vectratech.in**